odin-lang.org

Язык программирования Odin

Основы языка и базовые конструкции

Высокая производительность

Простота и ясность

Data-Oriented Design

odin-lang.org

История создания Odin

С чего всё началось и ключевые этапы развития



Июль 2016 года

Ginger Bill столкнулся с неудобствами C++. Изначально планировал препроцессор для C, но подход оказался тупиковым.

Было решено создать новый язык с нуля.

Ранние версии были похожи на Pascal

Проект быстро развился в самостоятельный язык.

Влияние языков

При проектировании опирался на:

- **Pascal** — философия
- **C** — системный подход
- **Go** — простота и практичность
- **GLSL** — обработка данных

Философия Odin

- Простота и читаемость кода
- Минимум способов сделать одно и то же
- Ортогональность концепций
- Фокус на преобразовании данных
- Проверенные временем идеи
- Компактная спецификация

odin-lang.org

Где применяется Odin?

Реальные продукты и области применения

Игры

- **CAT & ONION** — коммерческая 2D-игра, написанная на Odin + Raylib
- **Solar Storm** — тактическая sci-fi игра с собственным движком на Odin

Game Development

VFX и 3D

- **EmberGen** — симуляция огня, дыма и взрывов в реальном времени
- **GeoGen** — генерация ландшафтов и планет
- **LiquiGen** — симуляция жидкостей

JangaFX

Инструменты и ПО

- **Blick** — нативный нелинейный видеоредактор
- **ChiAha Digital Twin Toolkit** — моделирование производственных линий
- **Reaper Monitor** — системный мониторинг для macOS

Commercial / Production

`odin-lang.org`

Установка и запуск

Компилятор odin — один исполняемый файл: он и собирает, и запускает, и проверяет код

Как начать

- Скачать релиз `dev-YYYY-MM` с GitHub или собрать из исходников
- Добавить папку с `odin` в PATH
- Linux / macOS: нужен **clang** (линковка), Windows: **MSVC Build Tools**
- Редактор: VS Code + **ols** (language server)

examples/01_hello.odin

```
package main

import "core:fmt"

main :: proc() {
    fmt.println("Hellope!")
}
```

Основные команды

`odin run .` — собрать и запустить

`odin build . -o:speed` — релизная сборка

`odin check .` — только проверка типов



Один файл можно запустить как пакет: `odin run hello.odin -file`

odin-lang.org

Структура программы

Разбираем минимальный файл: пакет, импорт, константа и точка входа

examples/02_structure.odin

```odin
// Пакет = папка с .odin-файлами
package main

import "core:fmt" // пакет из коллекции core

/* Многострочный комментарий
   /* может быть вложенным */ */

GREETING :: "Hello, Odin!" // константа

main :: proc() {
    fmt.println(GREETING)
}
```

`package main`

Пакет = папка с .odin-файлами. Исполняемая программа — пакет `main`

`import "core:fmt"`

Коллекции библиотек: `base:`, `core:`, `vendor:`

`main :: proc()`

Точка входа. `::` объявляет константу — процедура тоже константа

`{ }` без `;`

Блоки — в фигурных скобках, точка с запятой не нужна

В Odin нет классов, исключений и сборщика мусора: программа — это **данные** (типы) и **процедуры**, которые их обрабатывают.

odin-lang.org

Переменные и константы

Odin статически типизирован, но выводит тип из значения

examples/03_variables.odin · строки 6–15

```
x: int          // объявление: x == 0
y: int = 10     // тип + значение
z := 20         // тип выводится: int
name := "Odin" // string

x = y + z       // присваивание
a, b := 1, 2
a, b = b, a     // множественное присваивание

fmt.println(x, name, a, b) // 30 Odin 2 1
```

examples/04_constants.odin · строки 5–6

```
PI :: 3.14159         // нетипизированная константа
MAX_USERS : int : 100 // типизированная константа
```

`x: T = v`

Переменная с явным типом. Без значения — нулевая инициализация

`x := v`

Тип выводится из значения. Самая частая форма

`NAME :: v` — константа

Значение известно на этапе компиляции и не меняется. Может быть типизированной: `MAX : int : 100`

Нулевые значения: `0`, `false`, `""`, `nil`. Мусора в памяти нет, если явно не попросить `x: int = ---`

odin-lang.org

Базовые типы и преобразования

Размер каждого типа явный, а преобразования между типами — только явные

Целые
`int` `uint`
`i8…i128`
`u8…u128` `byte`

Вещественные
`f16` `f32` `f64`

Логические
`bool`
`b8…b64`

Текст
`string` `rune`
`cstring`

Прочие
`rawptr` `any`
`typeid`

examples/05_types.odin · строки 6–11

```
age:    int    = 21
small:  u8     = 255
price:  f64    = 19.99
active: bool   = true
letter: rune   = 'O'
name:   string = "Odin"
```

examples/06_conversions.odin · строки 6–15

```
i: int = 7
f := f64(i) / 2      // 3.5   — явное преобразование
g := cast(f64)i      // 7     — то же самое
d := 9.99
n := int(d)          // 9     — дробная часть отбрасывается

// h: f64 = i        // ошибка: неявных преобразований нет

bits := transmute(u32)f32(1.0) // переинтерпретация битов
fmt.println(f, g, n, bits)     // 3.5 7 9 1065353216
```

odin-lang.org

Операторы и строки

База, которая понадобится в любой программе

examples/07_operators.odin · строки 6–13

```
a, b := 10, 3

fmt.println(a + b, a * b)     // 13 30
fmt.println(a / b)            // 3: целочисленное
fmt.println(-7 % 3)           // -1: остаток
fmt.println(-7 %% 3)          // 2: модуль
fmt.println(a > b && b != 0) // true
fmt.println(a ~ b)            // 9: XOR
```

examples/08_strings.odin · строки 7–17

```
name, age := "Anna", 21

fmt.println(name, "is", age)          // Anna is 21
fmt.printfln("%s is %d", name, age)   // как в C
fmt.printfln("%v: %v", name, age + 1) // Anna: 22

s := fmt.tprintf("%d%%", 42) // "42%"
fmt.println(len(s))          // 3 (байта)

n, ok := strconv.parse_int("123")
fmt.println(n, ok)           // 123 true
```

Арифметика

`/` для целых — целочисленное деление. `%` — остаток со знаком, `%%` — математический модуль. `~` — XOR

Строки и fmt

`string` — UTF-8 байты, `len` даёт байты. `printfln` = `printf` + перевод строки, `%v` — любое значение. `parse_int` возвращает `(value, ok)`

odin-lang.org

Условия: if

Скобки вокруг условия не нужны, фигурные скобки обязательны

examples/09_if.odin · строки 6–23

```
age := 20

if age >= 18 {
    fmt.println("Adult")
} else {
    fmt.println("Minor")
}

// init-оператор: x виден только внутри if/else
if x := age * 2; x > 30 {
    fmt.println("x =", x)
}

if age > 0 do fmt.println("one-liner") // do вместо {}

// if — оператор, а не выражение; для значений есть тернарный
status := "Adult" if age >= 18 else "Minor"
fmt.println(status)
```

Без скобок

`if x > 0 { ... }`. Однострочное тело можно записать через `do`

Init-оператор

`if x := f(); x > 0 {}` — переменная живёт только внутри if/else

Тернарный оператор

if — оператор, а не выражение. Для значения: `a if cond else b` или `cond ? a : b`

odin-lang.org

switch

Выбирает одну ветку — без break и без случайного «проваливания»

examples/10_switch.odin · строки 6–19

```
code := 404
switch code {
case 200:      fmt.println("OK")
case 404, 410: fmt.println("Not Found") // break не нужен
case 500..<600: fmt.println("Server Error")
case:          fmt.println("Unknown")   // default
}

age := 30
switch { // switch без условия = цепочка if-else
case age < 18: fmt.println("Minor")
case age < 65: fmt.println("Adult")
case:          fmt.println("Senior")
}
```

Что важно

- Выполняется только одна ветка, `break` не нужен
- Несколько значений: `case 404, 410:`
- Диапазоны: `case 500..<600:`
- `case:` — ветка по умолчанию

`fallthrough`

Явный переход в следующую ветку, если он действительно нужен

switch по `enum` и `union` проверяет, что все варианты разобраны (`#partial` отключает проверку)

odin-lang.org

for — единственный цикл

while, do-while и foreach в Odin записываются через одно ключевое слово

examples/11_for.odin · строки 6–15

```
for i := 0; i < 3; i += 1 { fmt.print(i, "") } // 0 1 2  (как в C)

j := 0
for j < 3 { j += 1 }        // как while

for i in 0..<5 { fmt.print(i, "") } // 0 1 2 3 4
for i in 1..=5 { fmt.print(i, "") } // 1 2 3 4 5

for { break }               // бесконечный цикл
fmt.println()
```

`0 ..< 5`	0	1	2	3	4	полуоткрытый: правая граница не входит
`1 ..= 5`	1	2	3	4	5	закрытый: правая граница входит

C-стиль: `for i := 0; i < n; i += 1`

Как while: `for cond { }`

Бесконечный: `for { }`

`break` / `continue`

Работают как в C; с метками можно выйти сразу из вложенного цикла: `break outer`

odin-lang.org

for in: обход коллекций

Один синтаксис для диапазонов, массивов, срезов, строк и map

examples/12_for_in.odin · строки 6–19

```
names := []string{"Ann", "Bob", "Max"}

for name in names     { fmt.println("Hello,", name) }
for name, i in names  { fmt.println(i, name) }

for r in "Odin"       { fmt.print(r, "") }  // по рунам
fmt.println()

#reverse for name in names { fmt.print(name, "") } // с конца
fmt.println()

nums := [3]int{1, 2, 3}
for &n in nums { n *= 10 }    // по ссылке: можно менять
fmt.println(nums)             // [10, 20, 30]
```

`for v, i in xs`

Значение и индекс. Для map — ключ и значение

Строки

Обход идёт по символам `rune` (UTF-8), а не по байтам

`#reverse for` — обход с конца

`for &v in xs` — по ссылке, элемент можно менять

odin-lang.org

Процедуры

Функции в Odin называются процедурами и объявляются как константы

examples/13_procedures.odin · строки 5–25

```
square :: proc(x: int) -> int {
    return x * x
}

// несколько результатов
div_mod :: proc(a, b: int) -> (int, int) {
    return a / b, a % b
}

// параметр со значением по умолчанию
greet :: proc(name: string, greeting := "Hello") {
    fmt.println(greeting, name)
}

main :: proc() {
    fmt.println(square(5))          // 25
    q, r := div_mod(17, 5)          // 3 2
    fmt.println(q, r)
    greet("Anna")                   // Hello Anna
    greet("Bob", greeting = "Hi")   // именованный аргумент
}
```

Как читается сигнатура

`square :: proc(x: int) -> int`

`square` — имя, `::` — константа

`x: int` — параметр, `-> int` — результат

Несколько результатов

`-> (int, int)` и `q, r := div_mod(17, 5)` — без кортежей и структур

Параметры

Значения по умолчанию и именованные аргументы. Параметры неизменяемы внутри процедуры

odin-lang.org

Перегрузка и процедуры как значения

Процедуры можно группировать, хранить в переменных и передавать как аргументы

examples/14_proc_values.odin · строки 5–20

```
// явная перегрузка: группа процедур под одним именем
to_str_int  :: proc(x: int)  -> string { return "int" }
to_str_bool :: proc(x: bool) -> string { return "bool" }
to_str :: proc{to_str_int, to_str_bool}

apply :: proc(xs: []int, f: proc(int) -> int) {
    for &x in xs { x = f(x) }
}

main :: proc() {
    fmt.println(to_str(1), to_str(true)) // int bool

    nums := []int{1, 2, 3}
    apply(nums, proc(x: int) -> int { return x * 2 }) // литерал процедуры
    fmt.println(nums) // [2, 4, 6]
}
```

Явная перегрузка

`proc{a, b}` собирает группу; компилятор выбирает вариант по типам аргументов

Тип процедуры

`proc(int) -> int` — обычный тип: его принимают параметры и хранят поля структур

Замыканий нет: литерал процедуры не захватывает локальные переменные окружения

odin-lang.org

Обработка ошибок

Исключений нет: ошибка — это обычное значение, которое возвращает процедура

examples/15_errors.odin · строки 6–25

```
parse_age :: proc(s: string) -> (age: int, ok: bool) {
    age = strconv.parse_int(s) or_return // при ok == false — выход
    return age, age >= 0
}

main :: proc() {
    if age, ok := parse_age("21"); ok {
        fmt.println("age:", age)
    }

    _, ok := parse_age("abc")
    fmt.println(ok) // false

    m := map[string]int{}
    defer delete(m)
    v := m["missing"] or_else -1 // значение по умолчанию
    fmt.println(v) // -1
}
```

`(value, ok)`

Последний результат — `bool` или `Error`: успех или ошибка

`or_return`

Если ошибка — сразу вернуть её из текущей процедуры

`or_else v` — значение по умолчанию

`or_continue` / `or_break` — для циклов

odin-lang.org

defer

Отложенное действие выполняется при выходе из текущей области видимости

examples/16_defer.odin · строки 5–15

```
main :: proc() {
    fmt.println("start")
    defer fmt.println("1: выполнится последним")
    defer fmt.println("2: выполнится раньше")

    {
        defer fmt.println("конец вложенного блока")
        fmt.println("внутри блока")
    }
    fmt.println("end")
}
```

Вывод программы

```
$ odin run 16_defer.odin -file
start
внутри блока
конец вложенного блока
end
2: выполнится раньше
1: выполнится последним
```

Порядок: обратный (LIFO)

Когда: в конце блока `{}`, а не процедуры

Зачем: `open` → `defer close` рядом

odin-lang.org

Массивы, срезы и динамические массивы

Три способа хранить последовательность элементов

examples/17_arrays.odin · строки 6–19

```
arr := [5]int{1, 2, 3, 4, 5} // фиксированный массив
arr[0] = 100

s := arr[1:4]                // срез: [2, 3, 4]
fmt.println(len(arr), s)     // 5 [2, 3, 4]

dyn: [dynamic]int            // динамический массив
defer delete(dyn)            // память освобождаем сами
append(&dyn, 10, 20, 30)
fmt.println(dyn, len(dyn))   // [10, 20, 30] 3

a := [3]f32{1, 2, 3}
b := [3]f32{4, 5, 6}
fmt.println(a + b, a * 2)    // [5, 7, 9] [2, 4, 6]
```

`[N]T` — массив

Фиксированный размер, хранится по значению (копируется)

`[]T` — срез

Указатель + длина: «окно» в чужие данные, `arr[1:4]`

`[dynamic]T`

Растёт через `append`. Память выделяется в куче, поэтому нужен `delete`

Массивы чисел складываются поэлементно: `a + b`

odin-lang.org

Словари: map

Хеш-таблица «ключ → значение» встроена в язык

examples/18_maps.odin · строки 5–22

```
main :: proc() {
    ages := make(map[string]int)
    defer delete(ages)

    ages["Anna"] = 21
    ages["Bob"]  = 25

    if age, ok := ages["Anna"]; ok {
        fmt.println("Anna:", age)
    }
    fmt.println("Max" in ages)    // false

    delete_key(&ages, "Bob")

    for name, age in ages {
        fmt.println(name, age)
    }
}
```

`make` / `delete`

map живёт в куче: создаём через `make`, освобождаем через `delete`

`v, ok := m[k]`

Есть ли ключ? Для отсутствующего ключа — нулевое значение

`k in m` — проверка, `delete_key(&m, k)` — удаление

Порядок обхода `for k, v in m` не гарантирован

odin-lang.org

Структуры

Пользовательские типы данных: поля без методов и наследования

examples/19_structs.odin · строки 5–23

```
Vec2 :: struct {
    x, y: f32,
}

Player :: struct {
    name:  string,
    pos:   Vec2,
    hp:    int,
}

main :: proc() {
    p := Player{name = "Hero", hp = 100} // pos = {0, 0}
    p.pos.x = 5

    q := Vec2{1, 2}              // позиционная запись
    fmt.println(p)
    fmt.println(q.x + q.y)       // 3
}
```

Объявление

`Name :: struct { поле: Тип, ... }` — это тоже константа, как и процедура

Литералы

Позиционно `Vec2{1, 2}` или по именам `Player{name = "Hero"}`; остальные поля — нули

Вывод: fmt печатает структуру целиком

```
Player{name = "Hero",
  pos = Vec2{x = 5, y = 0}, hp = 100}
3
```

odin-lang.org

enum и union

Перечисления и размеченные объединения вместо иерархий классов

examples/20_enums_unions.odin · строки 5–25

```
Direction :: enum { North, East, South, West }

Shape :: union { Circle, Rect }
Circle :: struct { r: f32 }
Rect   :: struct { w, h: f32 }

area :: proc(s: Shape) -> f32 {
    switch v in s {   // switch по типу
    case Circle: return 3.14 * v.r * v.r
    case Rect:   return v.w * v.h
    }
    return 0          // s == nil
}

main :: proc() {
    d := Direction.North
    d = .West                    // неявный селектор
    fmt.println(d)               // West

    fmt.println(area(Rect{2, 3})) // 6
}
```

enum

Именованные целые значения. `.West` — неявный селектор: тип известен из контекста

union

Значение одного из типов + метка, какой тип сейчас хранится. Нулевое значение — `nil`

`switch v in s`

Switch по типу: внутри ветки `v` уже нужного типа

odin-lang.org

Указатели

Адрес значения в памяти — без арифметики указателей

examples/21_pointers.odin · строки 5–21

```
Counter :: struct { value: int }

increment :: proc(c: ^Counter) {
    c.value += 1     // авто-разыменование для полей
}

main :: proc() {
    x := 10
    p: ^int = &x     // указатель на x
    p^ = 20          // разыменование
    fmt.println(x)   // 20

    c := Counter{}
    increment(&c)
    increment(&c)
    fmt.println(c.value) // 2
}
```



^T

тип «указатель на T»

&x / p^

взять адрес / прочитать значение

Передача по ссылке

Параметры неизменяемы, поэтому чтобы изменить значение, процедура принимает указатель. Поля доступны через точку: `c.value`

odin-lang.org

Память и аллокаторы

Сборщика мусора нет: память выделяется и освобождается явно

examples/22_memory.odin · строки 5–16

```
main :: proc() {
    n := new(int)          // выделить память под int
    defer free(n)          // освободить в конце области
    n^ = 42

    nums := make([]int, 5) // срез из 5 элементов в куче
    defer delete(nums)

    // временная память: всё освобождается разом
    msg := fmt.tprintf("n = %d, len = %d", n^, len(nums))
    fmt.println(msg)
    free_all(context.temp_allocator)
```

`new` → `free`

Один объект, возвращает указатель

`make` → `delete`

Срезы, динамические массивы, map

Неявный `context`

Каждая процедура получает `context` с аллокаторами: `context.allocator` — общий, `context.temp_allocator` — временный, чистится через `free_all`

Типичная пара: выделили память → сразу `defer` освобождение. Для поиска утечек в core есть **tracking allocator** (`core:mem`), который печатает неосвобождённые блоки.

odin-lang.org

Итераторы

Любая процедура, возвращающая (значение, bool), подходит для for in

examples/23_iterators.odin · строки 6–24

```
Countdown :: struct { n: int }

// итератор: процедура, последний результат которой — bool
countdown_next :: proc(it: ^Countdown) -> (value: int, ok: bool) {
    if it.n <= 0 do return 0, false
    it.n -= 1
    return it.n + 1, true
}

main :: proc() {
    it := Countdown{3}
    for v in countdown_next(&it) { fmt.print(v, "") } // 3 2 1
    fmt.println()

    text := "first\nsecond\nthird"
    for line in strings.split_lines_iterator(&text) {
        fmt.println(">", line)
    }
}
```

Протокол

Итератор вызывается на каждом шаге; цикл идёт, пока последний результат — `true`

Состояние

Хранится в структуре, которую передаём по указателю `&it`

В библиотеке

`strings.split_lines_iterator`, `split_iterator` и др.

odin-lang.org

Области видимости

Имя доступно внутри блока, где оно объявлено, и во вложенных блоках

examples/24_scope.odin · строки 5–24

```
x := 10 // переменная уровня пакета

example :: proc() {
    y := 20                  // видна в процедуре
    if x > 0 {
        z := 30              // видна только в блоке if
        fmt.println(x + y + z) // 60
    }
    // fmt.println(z)        // ошибка: z не объявлена

    {
        y := 99 // затенение (odin -vet запрещает)
        fmt.println(y) // 99
    }
    fmt.println(y) // 20
}

main :: proc() { example() }
```

package scope · **x**

procedure scope · **y**

if block · **z**

здесь видны x, y и z

Повторное объявление в том же блоке — ошибка. Затенение во вложенном блоке разрешено, но `-vet` его запрещает

odin-lang.org

Ввод и вывод в консоль

Вывод — пакет fmt, ввод — чтение stdin через буферизованный reader

examples/25_console.odin · строки 9–27

```
main :: proc() {
    r: bufio.Reader
    bufio.reader_init(&r, os.to_reader(os.stdin))
    defer bufio.reader_destroy(&r)

    fmt.print("Enter your name: ")
    line, _ := bufio.reader_read_string(&r, '\n', context.temp_allocator)
    name := strings.trim_space(line)

    fmt.print("Enter your age: ")
    line, _ = bufio.reader_read_string(&r, '\n', context.temp_allocator)
    age, ok := strconv.parse_int(strings.trim_space(line))
    if !ok {
        fmt.println("Not a number!")
        return
    }

    fmt.printfln("Hello, %s!", name)
    fmt.printfln("Next year: %d", age + 1)
```

Терминал

```
Enter your name: Anna
Enter your age: 21
Hello, Anna!
Next year: 22
```

Что важно

- `reader_read_string` возвращает строку вместе с `\n`, поэтому `trim_space`
- Число из строки — `strconv.parse_int`, проверяем `ok`
- `fmt.eprintln` пишет в stderr

odin-lang.org

Чтение файла

Читаем файл целиком и обходим его по строкам итератором

examples/26_read_file.odin · строки 7–18

```odin
main :: proc() {
    data, err := os.read_entire_file("numbers.txt", context.allocator)
    if err != nil {
        fmt.eprintln("cannot read file:", err)
        return
    }
    defer delete(data)

    text := string(data)
    for line in strings.split_lines_iterator(&text) {
        fmt.println(line)
    }
}
```

numbers.txt

```
10
75
60
42
90
```

read_entire_file

Возвращает `([]byte, Error)`. Память выделяет переданный аллокатор, поэтому `defer delete(data)`

`string(data)` — без копирования

`err != nil` — файл не прочитан

odin-lang.org

Запись в файл

Открываем файл, пишем через fmt и закрываем через defer

examples/27_write_file.odin · строки 6–19

```
main :: proc() {
    f, err := os.create("result.txt")
    if err != nil {
        fmt.eprintln("cannot create file:", err)
        return
    }
    defer os.close(f)

    fmt.fprintln(f, "Hello, Odin!")
    fmt.fprintfln(f, "Answer: %d", 42)

    // или одной строкой:
    _ = os.write_entire_file("short.txt", "Hellope!\n")
}
```



os.create

Создаёт (или очищает) файл и возвращает `(^os.File, Error)`

fmt.fprintln

Те же функции fmt, но первым аргументом — файл

Короткий путь: `os.write_entire_file(path, text)` записывает всё содержимое одним вызовом

odin-lang.org

Итоговый пример: чтение → обработка → запись

examples/28_final.odin · строки 8–27

```
main :: proc() {
    data, err := os.read_entire_file("numbers.txt", context.allocator)
    if err != nil do return
    defer delete(data)

    sum, count, passed := 0, 0, 0
    text := string(data)
    for line in strings.split_lines_iterator(&text) {
        n := strconv.parse_int(strings.trim_space(line)) or_continue
        sum += n
        count += 1
        if n >= 60 do passed += 1
    }
    average := f64(sum) / f64(count)

    out, _ := os.create("result.txt")
    defer os.close(out)
    fmt.fprintfln(out, "Average: %.2f", average)
    fmt.fprintfln(out, "Passed: %d of %d", passed, count)
}
```

- os
- defer
- iterator
- or_continue
- f64()
- fprintfln

numbers.txt (по числу в строке)

```
10  75  60  42  90
```

result.txt

```
Average: 55.40
Passed: 3 of 5
```

Некорректные строки пропускаются через `or_continue`.

`odin-lang.org`

Итоги первой презентации

Что уже умеем после базового знакомства с Odin

собрать и запустить	объявлять := и ::	работать с типами	писать if и switch
использовать for / for in	писать процедуры	обрабатывать ошибки	освобождать через defer
массивы, срезы, map	struct, enum, union	указатели и память	консоль и файлы

Что рассмотрим дальше

- пакеты, `@(private)`, `when` и условная компиляция
- аллокаторы подробно, `context`, tracking allocator
- `bit_set`, `distinct`, параметрический полиморфизм `$T`
- SOA-структуры, матрицы, связь с C через `foreign`

Вопросы?

`odin-lang.org/docs`